

FPGA-Based Hardware Acceleration Using PYNQ-Z2

Vineeth C Johnson ^{1a*}, Jyoti Bali ^{2b}, Shilpa Tanvashi ^{3c}, C B Kolanur ^{4d}

Department of Automation & Robotics, KLE Technological University, Hubballi, Karnataka, India

vineethcjohnson@gmail.com

Abstract—A study on the FPGA development board PYNQ-Z2 for hardware acceleration is presented in this research paper. The experiment accelerates the tasks of optical character recognition (OCR) and image recognition using the FPGA on PYNQ-Z2. The output results on hardware acceleration (Processing system (PS) and Programmable Logic (PL)) are compared with the output results obtained while executing the same tasks on the Arm processor (Processing System (PS)) alone. In this experiment, a Long short-term memory (LSTM) neural network is used to implement OCR, and a Binarized neural network (BNN) is used to implement image recognition. LSTM and BNN here are quantized to reduce memory usage while implementing them on PYNQ-Z2.

Keywords—FPGA; Hardware acceleration; Image recognition; PYNQ

I. INTRODUCTION

A field-programmable gate array (FPGA) comprises a matrix of programmable logic, memory, and digital signal processing (DSP) elements. These elements are dispersed among a grid of programmable chip I/O blocks coupled to adjustable routing wires. Any digital circuit's features can be programmed to operate on this versatile and programmable fabric. FPGAs are programmed using hardware description languages such as Verilog and Very High-Speed Integrated Circuit (VHSIC) Hardware Description Language (VHDL). Additionally, frameworks are available for using popular software programming languages like C/C++ for Xilinx FPGAs or OpenCL for Intel FPGAs. Recent initiatives aim to use FPGAs with domain-specific languages. In FPGA binaries, referred to as bitstreams, programs are compiled. Before creating a netlist of basic FPGA components, the HDL compiler converts the program statements (memory, logic, and arithmetic). The netlist is then assigned to physical components in the FPGA fabric, and the routing wires that should be utilized to connect them are determined. The name of the latter procedure is called Place and Route. [1]

There are three different sorts of modules in the typical FPGA architecture. They are configurable logic blocks (CLB), switch matrixes and interconnection wires, and I/O blocks or pads. The fundamental FPGA architecture consists of two-dimensional arrays of logic blocks that can be connected in any order by the user. Digital inputs, outputs, and logic are all included in a CLB (Configurable Logic Block). It carries out user logic. Interconnects give the logic blocks direction so they can implement the user logic. The switch matrix offers to

switch between interconnects according to the logic. I/O pads are utilized to connect many applications to the outside world. The MUX (Multiplexer), D flip-flop, and LUT are the components of a logic block. The MUX is utilized for the logic selection. The LUT performs the combinational logical operations, and the D flip-flop stores the LUT's output. The core component of the FPGA is the look-up table-based function generator. ASIC prototyping, bioinformatics, digital signal processing, filtering, device controllers, medical imaging, image processing, voice recognition, communication encoding, cryptography, and many other applications are applications where an FPGA is specifically used [2-6]. The following section briefly describes the PYNQ-Z2 FPGA board and the experimental setup used in this experiment.

II. MATERIALS AND METHODS

A. PYNQ-Z2 FPGA board

The FPGA development board used in this research is PYNQ-Z2. A new open-source framework called PYNQ allows embedded programmers to explore the potential of Xilinx ZYNQ SoCs without having to create programming logic circuits. PYNQ-Z2 is an FPGA development board based on the ZYNQ XC7Z020 FPGA, having 13,300 logic slices, each of which has four 6-input LUTs and eight flip-flops. Other features include 220 DSP slices, 512MB DDR3, 16MB Quad SPI-Flash, and 220 DSP slices. ZYNQ's sophisticated ARM CPU and programmable logic allow designers to create embedded systems that are more powerful.

Additionally, Python may program the SoCs, and code can be generated and tested directly on the PYNQ-Z2. The programmable logic circuits are imported as hardware libraries and implemented using APIs in a similar way the software libraries are imported. The PYNQ Z2 board requires a computer, a micro SD card flashed with the PYNQ image, an Ethernet cable, a USB cable, and an Internet router (only when the board requires internet access) for its functioning. Fig. 1 shows the block diagram of the PYNQ-Z2 setup.

Once the board is booted up, it is possible to program it using Jupyter Notebook using Python. The board's ARM processor powers the notebook server [7,8]. PYNQ Z2 supports the PYNQ framework, which can create high-performance applications with hardware parallel execution, hardware accelerated algorithms, high bandwidth IO and low latency

control. The implementation of PYNQ-Z2 for hardware acceleration is explained in the following section.

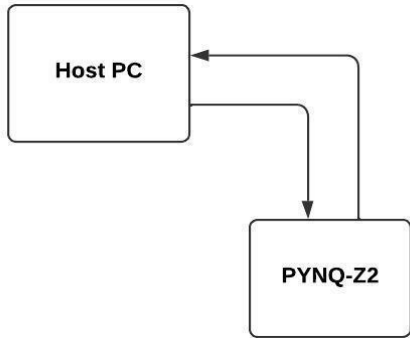


Fig 1. Block diagram of the PYNQ-Z2 setup

III. IMPLEMENTATION

There is a need to compare the performance between the Arm CPU and its FPGA hardware-accelerated version on the PYNQ-Z2 board for the two identified tasks: optical character recognition (OCR) and image recognition. Both tasks are executed by running using the PS alone and by using both PS and PL with the workload distributed between them. The Arm processor runs the software, and the FPGA accelerates the processor. The following are tasks performed to demonstrate hardware acceleration using PYNQ-Z2 field-programmable gate array.

A. Optical character recognition using LSTM

Long short-term memory (LSTM) is a type of artificial neural network used in deep learning and artificial intelligence and has feedback connections. Applications for LSTM include optical character recognition, speech recognition, machine translation, networked, robot control, and handwriting analysis. [9]. Printed or written text by hand is transformed into machine-encoded text using optical character recognition. It is a component of numerous procedures, including machine translation, mining text, and converting it to speech. The package for Quantized LSTM on PYNQ is installed in the PYNQ Jupyter Notebook directory from the Xilinx/LSTM-PYNQ GitHub repository.

TABLE 1. Images of the printed texts

Text 1	Seek access to aggregated patient
Text 2	Policymakers seek data to monitor
Text 3	many, the process has to be described in
Text 4	instead of a series over time, to obtain
Text 5	patterns are that less time is spent pro

The package contains the overlay that performs optical character recognition (OCR) of old German Fraktur and plain text. When the OCR model is instantiated, the proper bitstream is automatically downloaded to the device, and the weights from the plain text dataset are loaded. [10].

Table 1 shows the images of the text loaded to the Jupyter Notebook on PYNQ Z2 to perform OCR. The flowchart of the Python code in Fig 2 illustrates the FPGA-accelerated LSTM implementation on PYNQ to perform OCR on plain text.

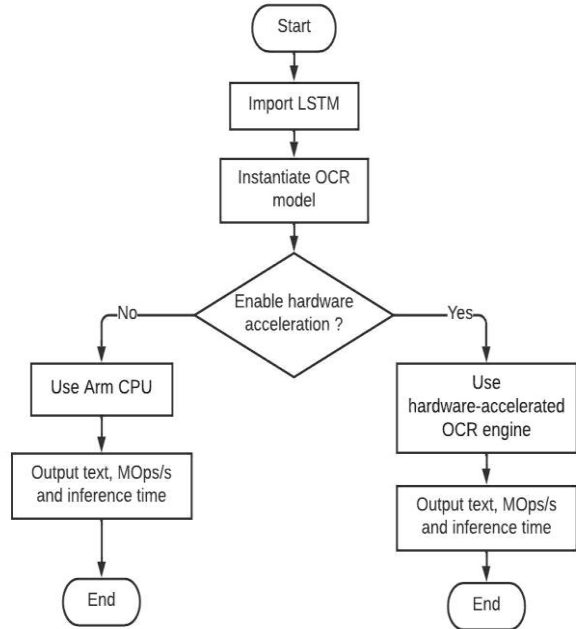


Fig 2. Flowchart of the Python code for OCR

The Quantized LSTM neural network recognizes all the characters from the printed text images in Table 1 and the profiling is done in terms of inference time and memory operations per second (MOps/s).

B. Image recognition using BNN

Networks containing binary weights and activations at runtime are known as binary neural networks (BNN). Although the gradients and real weights are recorded with full precision, these weights and activations are used to compute them during training. Hence the network can be trained efficiently on less resource-rich devices [11]. Fig 3 shows the input images run on the BNN trained using the CIFAR dataset of 10 classes. The classes include an aero-plane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck [12].



Image 1



Image 2



Image 3



Image 4



Image 5

Fig 3. Images run on the BNN

The package for Quantized BNN on PYNQ is installed in the PYNQ Jupyter Notebook directory from Xilinx/BNN-PYNQ GitHub repository. According to the FINN Paper, two different network topologies, CNV and LFC, are included in this package, and multiple implementations are available supporting additional precision for weights and activations. [12]. To study deep neural network inference on FPGAs, Xilinx Research Labs has developed the experimental framework known as FINN. It focuses on creating designs in the dataflow style that are unique to each network and is specifically targeted towards quantized neural networks. It is a tool for exploring the design space of DNN inference accelerators on FPGAs. [13]. Fig 4 illustrates the flowchart of the Python code for implementing BNN on PYNQ for image recognition.

The Quantized BNN recognizes all the images in fig 3 and correctly classify them as Aero-plane, Automobile, Bird, Cat, and Deer. The profiling is done regarding inference time and classification rate (images/second). The following section explains the results obtained on performing OCR using Quantized LSTM and image recognition using Quantized BNN.

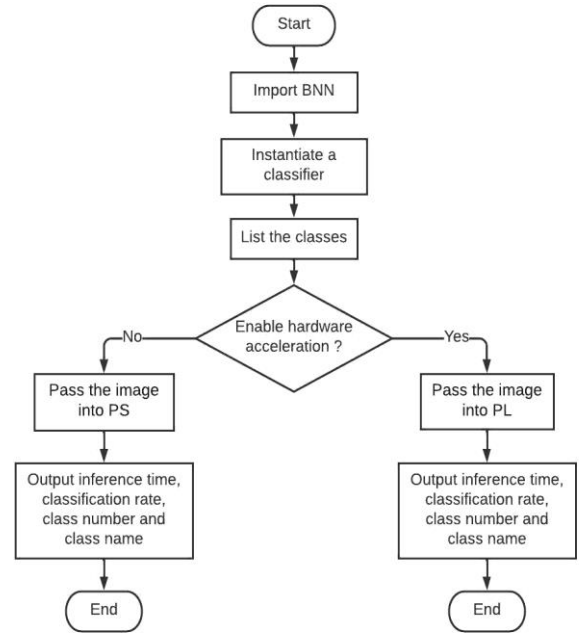


Fig 4. Flowchart of the Python code for image recognition

IV. RESULTS

Thus on PYNQ Z2, profiling the performance of the tasks OCR and image recognition when executed using an FPGA accelerator and when executed using the Arm processor alone and comparing them shows that using an FPGA accelerator helps in faster inference and execution without any loss in accuracy. Tables 2 and 3 analyze and summarize the results obtained from this experiment.

PS - Processing System (Arm processor)

PL - Programmable Logic (FPGA)

TABLE 2. OCR results

Task	Inference time (ms)		MOps/s	
	PS	PS + PL	PS	PS + PL
Text 1	56699.66	2.288	2.529	62668.15
Text 2	60245.23	2.453	2.532	62186.83
Text 3	72230.27	2.957	2.530	61820.76
Text 4	83691.15	3.376	2.528	62678.40
Text 5	79778.88	3.264	2.529	61816.47

From the results shown in Table 2, it is evident that the OCR performance can be boosted using FPGA as the hardware accelerator. All the characters present in the images

of the printed text loaded to the Python code in the PYNQ Jupyter Notebook were correctly recognized by the Quantized LSTM. The inference time decreased by 99.996 %, and the MOPs/s increased by 99.996 % on accelerating the PS using the PL compared to performing OCR using the PS alone on PYNQ-Z2.

TABLE 3. Image recognition results

Task	Inference time (μs)		Classification rate (images/second)	
	PS	PS + PL	PS	PS + PL
Image 1	1587984	1582	0.63	632.11
Image 2	1588104	1583	0.63	631.71
Image 3	1588113	1583	0.63	631.71
Image 4	1588634	1584	0.63	631.31
Image 5	1589870	1582	0.63	632.11

From the results shown in Table 3, it is evident that the image recognition performance can be boosted using FPGA as the hardware accelerator. All the images loaded to the Python code in the PYNQ Jupyter Notebook were correctly recognized and classified by the Quantized BNN. The inference time decreased by 99.9%, and the MOPs/s increased by 99.9 % on accelerating the PS using the PL compared to performing image recognition using the PS alone on PYNQ-Z2.

V. CONCLUSION

This research, along with hands-on with the PYNQ-Z2 board, has provided fundamental knowledge on accelerating the tasks of OCR and image recognition using the ZYNQ FPGA on PYNQ-Z2. Various tasks can be accelerated using the FPGA on PYNQ-Z2, including video processing, real-time object detection, and several other embedded applications. Thus, FPGA-based hardware acceleration is highly efficient.

REFERENCES

[1] M. Owaida, G. Alonso, L. Fogliarini, A. Hock-Koon, and P.-E. Melet, "Lowering the latency of data processing pipelines through FPGA based hardware acceleration," *Proceedings of the VLDB Endowment*, vol. 13, no.1, pp. 71–85, September 2019, doi: <https://doi.org/10.14778/3357377.3357383>.

[2] J. Bali, Arpitha H, N. Anushree, and A. C. Giriyaapur, "Power-efficient Strategies for Sensing in Autonomous Mobile Robots, a critical requirement of I4.0 standard," vol. 1187, no. 1, pp. 012007–012007, Sep. 2021, doi: <https://doi.org/10.1088/1757-899x/1187/1/012007>.

[3] A. C. Nilavar, J. Bali, A. S. Huggi, A. C. Giriyaapur, and A. L. K, "Implementation of a Navigational Path Planning Algorithm for an Autonomous Mobile Robot," Aug. 2021, doi: <https://doi.org/10.1109/indiscon53343.2021.9582210>.

[4] A. S. Huggi, A. C. Nilavar, J. Bali, A. C. Giriyaapur, and G. K. Ashwini, "Implementation of Sensor Fusion for a Mobile Robot Application," Aug. 2021, doi: <https://doi.org/10.1109/rteict52294.2021.9573730>.

[5] Johnson, V. C., Bali, J. S., Kolanur, C. B., Tanwashi, S. (2022). Industry 4.0: Intelligent Quality Control and Surface Defect Detection. 3C Empresa. Investigación y pensamiento crítico, 11(2), 186-192. <https://doi.org/10.17993/3cemp.2022.110250.186-192>.

[6] "Know about FPGA Architecture and their Applications," *ElProCus - Electronic Projects for Engineering Students*, Sep. 18, 2014. <https://www.elprocus.com/fpga-architecture-and-applications/>.

[7] "PYNQ-Z2 Development Board," [www.dfrobot.com](https://www.dfrobot.com/product-1838.html). <https://www.dfrobot.com/product-1838.html>.

[8] "Jupyter Notebooks — Python productivity for Zynq (Pynq) v1.0," [pynq.readthedocs.io](https://pynq.readthedocs.io/en/v2.3/jupyter_notebooks.html). https://pynq.readthedocs.io/en/v2.3/jupyter_notebooks.html

[9] Wikipedia Contributors, "Long short-term memory," *Wikipedia*, Nov. 22, 2018. https://en.wikipedia.org/wiki/Long_short-term_memory.

[10] V. Rybalkin, A. Pappalardo, M. M. Ghaffar, G. Gambardella, N. Wehn, and M. Blott, "FINN-L: Library Extensions and Design Trade-off Analysis for Variable Precision LSTM Networks on FPGAs," *arXiv.org*, Jul. 11, 2018. <https://arxiv.org/abs/1807.04093>.

[11] "AI & Machine Learning," *Intel*. <https://www.intel.com/content/www/us/en/developer/articles/technical/binary-neural-networks.html>.

[12] Y. Umuroglu et al., "FINN," *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, Feb. 2017, doi: <https://doi.org/10.1145/3020078.3021744>.

[13] "FINN," [finn](https://xilinx.github.io/finn). <https://xilinx.github.io/finn>.